

## Chapitre 2 : Mise en Œuvre : Déploiement et Analyse des Performances

Ce chapitre couvre les deux volets de la mise en œuvre pratique :

- 1) Le déploiement des frameworks d'inférence sur appareils Android réels,
- 2) L'analyse quantitative des performances mesurées.

Il documente aussi l'ensemble du pipeline, des commandes d'installation jusqu'aux résultats de benchmark.

### Partie 1 : Déploiement via llama.cpp (Termux / UserLand)

Cette première partie détaille le déploiement de llama.cpp directement sur smartphone Android, sans passer par un service cloud ni par un SDK propriétaire. Elle couvre l'installation via Termux, la compilation du moteur d'inférence, le téléchargement d'un modèle quantifié et les premières mesures de performance. Cette voie constitue la référence la plus portable et la plus documentée pour l'inférence LLM embarquée, indépendamment du fabricant ou du niveau de gamme de l'appareil.

#### 1.1 Installation sur Android via Termux

Cette section décrit l'installation complète de llama.cpp sur un smartphone Android quelconque, du prérequis matériel jusqu'au dépannage des erreurs les plus fréquentes. Termux sert d'environnement Linux userland, sans nécessiter de root ni de modification du système. L'ensemble de la procédure est reproductible en une trentaine de minutes, hors téléchargement du modèle.

**1.1.1 Prérequis matériels** Avant de lancer l'installation, il convient de vérifier que l'appareil dispose de ressources suffisantes pour compiler et exécuter llama.cpp. Le tableau suivant distingue une configuration minimale, suffisante pour un modèle 1B quantifié, d'une configuration recommandée permettant d'exploiter des modèles plus grands avec davantage de marge mémoire.

| Critère        | Minimum          | Recommandé           |
|----------------|------------------|----------------------|
| RAM            | 6 Go             | 12 Go                |
| Stockage libre | 5 Go             | 10 Go                |
| SoC            | ARM64 quelconque | Snapdragon 8 Gen 2/3 |
| Android        | 7.0+             | 12+                  |

**Tableau 2.1** : Prérequis matériels pour l'installation via Termux

**1.1.2 Installation de Termux** L'installation se déroule entièrement dans Termux, sans accès root. Les étapes suivantes couvrent le téléchargement de l'application, la mise à jour des dépôts de paquets, l'installation des outils de compilation et la vérification de l'architecture processeur avant de poursuivre.

**1. Télécharger Termux depuis F-Droid** (pas le Play Store, version obsolète) :

<<https://f-droid.org/packages/com.termux/>>

2. Ouvrir Termux et mettre à jour les paquets. Termux utilise son propre gestionnaire de paquets (pkg), indépendant de celui du système Android. Avant toute installation, il est nécessaire de synchroniser la liste des paquets disponibles et de mettre à jour ceux déjà installés, afin d'éviter des conflits de versions lors de la compilation à venir. Cette étape est rapide (moins d'une minute) et ne nécessite pas de connexion très stable, le volume téléchargé restant faible :

```
pkg update && pkg upgrade -y
```

3. Installer les dépendances de compilation. La compilation de llama.cpp nécessite un compilateur C/C++, l'outil de génération CMake, ainsi que Git pour récupérer le code source et Python pour certains scripts auxiliaires du projet. La commande suivante installe l'ensemble de ces dépendances en une seule opération.

```
pkg install -y git cmake clang make python
```

4. Vérifier l'architecture du processeur. llama.cpp doit être compilé pour l'architecture ARM64 (aarch64), qui équipe la quasi-totalité des smartphones Android actuels. Cette vérification permet de s'assurer que l'appareil est compatible avant d'investir du temps dans une compilation vouée à l'échec sur une architecture non supportée. Si la commande renvoie autre chose que aarch64 (par exemple armv7l), la compilation devra être adaptée, voire n'est pas possible sur cet appareil :

```
uname -m
```

```
# Attendu : aarch64
```

**1.1.3 Compilation de llama.cpp** Une fois les dépendances installées, l'étape suivante consiste à récupérer le code source de llama.cpp et à le compiler localement, directement sur le smartphone. La compilation active le support des instructions ARM NEON, qui accélèrent significativement les calculs matriciels sur les processeurs mobiles.

La commande `cmake` configure la compilation en mode Release (optimisations activées) et active l'option `GGML_NATIVE` pour que le compilateur détecte et exploite automatiquement les extensions vectorielles du processeur cible. Le support OpenMP est désactivé car il n'apporte pas de gain significatif sur les architectures mobiles testées. La compilation elle-même mobilise l'ensemble des cœurs disponibles via `-j$(nproc)`, ce

qui réduit sensiblement le temps total.

La dernière commande vérifie que le binaire llama-cli a bien été généré :

```
# Cloner le dépôt
git clone https://github.com/ggml-org/llama.cpp
cd llama.cpp
# Compiler avec support ARM NEON (optimisation mobile)
cmake -B build \
-DMAKE_BUILD_TYPE=Release \
-DGGML_NATIVE=ON \
-DGGML_OPENMP=OFF
cmake --build build --config Release -j$(nproc)
# Vérifier la compilation
./build/bin/llama-cli --version
```

**1.1.4 Téléchargement du modèle** Une fois llama.cpp compilé, il faut télécharger un modèle au format GGUF, le format de fichier unique et portable utilisé par ce framework. Trois options sont proposées ci-dessous, chacune représentant un compromis différent entre taille, qualité et besoin en RAM.

Les modèles sont téléchargés directement depuis Hugging Face au format GGUF quantifié Q4\_K\_M, un bon compromis entre taille et qualité pour un déploiement mobile. Gemma 2 2B constitue le choix recommandé par défaut pour son équilibre entre performance et empreinte mémoire, tandis que LLaMA 3.2 3B offre de meilleures capacités de raisonnement au prix d'une empreinte plus importante. LLaMA 3.2 1B reste l'option la plus légère et convient aux appareils disposant de moins de 6 Go de RAM :

```
# Créer un dossier pour les modèles
mkdir -p ~/models && cd ~/models
# Option 1 : Gemma 2 2B (recommandé, open source Google, bon équilibre)
# Taille : ~1,6 Go (Q4_K_M)
curl -L -o gemma-2-2b-it-q4_k_m.gguf \
"https://huggingface.co/bartowski/gemma-2-2b-it-GGUF/resolve/main/gemma-2-2b-it-Q4_K_M.gguf"
# Option 2 : LLaMA 3.2 3B (Meta, très bon raisonnement)
# Taille : ~2,0 Go (Q4_K_M)
curl -L -o llama-3.2-3b-instruct-q4_k_m.gguf \
"https://huggingface.co/bartowski/Llama-3.2-3B-Instruct-GGUF/resolve/main/Llama-3.2-3B-Instruct-Q4_K_M.gguf"
# Option 3 : LLaMA 3.2 1B (le plus léger, pour appareils <6 Go RAM)
# Taille : ~0,8 Go (Q4_K_M)
curl -L -o llama-3.2-1b-instruct-q4_k_m.gguf \
"https://huggingface.co/bartowski/Llama-3.2-1B-Instruct-GGUF/resolve/main/Llama-3.2-1B-Instruct-Q4_K_M.gguf"
```

**1.1.5 Première inférence** Le binaire llama-cli, généré lors de la compilation, permet de lancer une inférence directement en ligne de commande, sans interface graphique. Deux

modes d'utilisation sont illustrés ci-dessous : un test ponctuel non-interactif, puis un mode conversationnel avec historique.

Le premier appel exécute une inférence unique à partir d'un prompt fixe, avec un nombre de tokens de sortie limité à 200 et une température de 0,7 favorisant une certaine diversité dans la réponse sans verser dans l'incohérence. Le second appel active le mode interactif (-i) avec le template de conversation propre à Gemma, une fenêtre de contexte de 2048 tokens et jusqu'à 512 tokens générés par tour de parole. Ces deux invocations couvrent les deux cas d'usage les plus courants d'un déploiement conversationnel embarqué :

```
cd ~/llama.cpp
# Test rapide (non-interactif)
./build/bin/llama-cli \
-m ~/models/gemma-2-2b-it-q4_k_m.gguf \
-p "Qu'est-ce que l'intelligence artificielle ?" \
-n 200 \
--temp 0.7
# Mode chat interactif
./build/bin/llama-cli \
-m ~/models/gemma-2-2b-it-q4_k_m.gguf \
-i \
--chat-template gemma \
-n 512 \
--temp 0.7 \
-c 2048
```

### Paramètres importants :

| Paramètre     | Description                 | Valeur recommandée mobile      |
|---------------|-----------------------------|--------------------------------|
| -n            | Nombre de tokens à générer  | 256-512                        |
| -c            | Taille du contexte (tokens) | 1024-2048                      |
| -temp         | Température (créativité)    | 0.7                            |
| -t            | Nombre de threads CPU       | \$(nproc) ou 4                 |
| -n-gpu-layers | Couches sur GPU             | 0 (CPU uniquement sur Android) |

**Tableau 2.2 :** Paramètres d'inférence llama.cpp recommandés pour mobile

**1.1.6 Mesure des performances** Au-delà du fonctionnement qualitatif, il est essentiel de mesurer objectivement le débit d'inférence et la consommation mémoire de l'appareil. llama.cpp fournit un outil de benchmark intégré permettant de quantifier séparément la phase de **prefill** (traitement du prompt) et la phase de **decode** (génération token par token).

L'outil llama-bench exécute une série de mesures répétées (-r 5) sur un prompt de 512 tokens en prefill et 128 tokens en decode, puis calcule une moyenne représentative en atténuant le bruit lié aux variations ponctuelles de charge système. Ces deux métriques

sont complémentaires : le prefill reflète la puissance de calcul brute (compute-bound), tandis que le decode est davantage limité par la bande passante mémoire (memory-bound). Sur un Snapdragon 8 Gen 3, les valeurs obtenues se situent typiquement entre 25 et 35 tokens/s en prefill et entre 12 et 18 tokens/s en decode :

```
# Benchmark intégré llama.cpp
./build/bin/llama-bench \
-m ~/models/gemma-2-2b-it-q4_k_m.gguf \
-p 512 \
-n 128 \
-r 5
# Résultat attendu sur Snapdragon 8 Gen 3 :
# pp512 : ~25-35 tok/s (prefill – traitement du prompt)
# tg128 : ~12-18 tok/s (decode – génération token par token)
```

En complément du benchmark de débit, un suivi de la consommation mémoire en temps réel permet de détecter d'éventuels dépassements de RAM (OOM) avant qu'ils ne provoquent l'arrêt brutal du processus. La commande suivante interroge périodiquement l'état de la mémoire système et affiche la RAM utilisée toutes les deux secondes, dans un second terminal Termux exécuté en parallèle de l'inférence :

```
# Monitoring mémoire en temps réel (dans un second terminal Termux)
while true; do
free -m | grep Mem | awk '{printf "RAM utilisée: %d Mo / %d Mo\n", $3, $2}'
sleep 2
done
```

**1.1.7 Dépannage fréquent** L'installation via Termux peut échouer pour plusieurs raisons liées aux contraintes spécifiques de l'environnement Android. Le tableau suivant recense les erreurs les plus fréquemment rencontrées au cours de ce PIR, leur cause probable et la solution appliquée.

| Erreur                                | Cause                  | Solution                          |
|---------------------------------------|------------------------|-----------------------------------|
| SIGKILL pendant l'inférence           | OOM (manque de RAM)    | Passer au modèle 1B ou réduire -c |
| llama_model_load: error loading model | Fichier GGUF corrompu  | Re-télécharger, vérifier MD5      |
| Performances très lentes              | Throttling thermique   | Pause 5 min, ventiler l'appareil  |
| pkg: command not found                | Termux non mis à jour  | pkg update && pkg upgrade         |
| cmake: not found                      | Dépendances manquantes | pkg install cmake clang           |

**Tableau 2.3** : Erreurs fréquentes et solutions lors de l'installation Termux

## 1.2 Installation sur PC Linux/Mac (développement)

Bien que ce PIR cible principalement le déploiement mobile, la compilation sur poste de travail (Linux ou Mac) reste utile.

Les étapes de compilation sont proches de celles utilisées sous Termux, à quelques options près liées à l'accélération matérielle disponible sur chaque plateforme.

**1.2.1 Linux (Ubuntu/Debian)** Sur une distribution Linux de bureau, l'installation des dépendances passe par le gestionnaire de paquets APT plutôt que par pkg. La compilation reste similaire à celle effectuée sous Termux, avec les mêmes optimisations natives activées.

La commande `apt install` installe les mêmes outils de compilation que sous Android (Git, CMake, un compilateur C++ et OpenMP). La compilation utilise ensuite l'option `GGML_NATIVE` pour adapter automatiquement le code aux instructions vectorielles du processeur hôte (AVX2 sur la plupart des CPU Intel/AMD récents), ce qui offre des performances nettement supérieures à un smartphone, du fait de la puissance de calcul et du refroidissement actif disponibles sur un PC :

```
# Dépendances
sudo apt update && sudo apt install -y git cmake build-essential libgomp1
# Cloner et compiler
git clone https://github.com/ggml-org/llama.cpp && cd llama.cpp
cmake -B build -DCMAKE_BUILD_TYPE=Release -DGGML_NATIVE=ON
cmake --build build --config Release -j$(nproc)
```

**1.2.2 Mac (Apple Silicon)** Sur Mac équipé d'une puce Apple Silicon (M1/M2/M3), `llama.cpp` peut exploiter le GPU intégré via l'API Metal, ce qui accélère significativement l'inférence par rapport à une exécution CPU seule. La procédure d'installation repose sur Homebrew plutôt que sur un gestionnaire de paquets Linux.

L'option `GGML_METAL=ON` active le support du GPU Apple lors de la compilation, ce qui permet ensuite de décharger tout ou partie des couches du modèle sur le GPU via le paramètre `-n-gpu-layers`. Sur une puce M1/M2/M3, positionner cette valeur à 99 revient à exécuter la quasi-totalité du modèle sur GPU, ce qui réduit fortement la latence par rapport à une inférence CPU pure. Cette configuration sert de référence de performance haute lors des comparaisons multi-plateformes de la section 1.4 :

```
# Homebrew + compilation avec Metal (GPU Apple)
brew install cmake
git clone https://github.com/ggml-org/llama.cpp && cd llama.cpp
cmake -B build -DCMAKE_BUILD_TYPE=Release -DGGML_METAL=ON
cmake --build build --config Release -j$(sysctl -n hw.logicalcpu)
# Sur Mac M1/M2/M3 : ajouter --n-gpu-layers 99 pour utiliser le GPU Metal
./build/bin/llama-cli -m ~/models/gemma-2-2b-it-q4_k_m.gguf \
--n-gpu-layers 99 -i --chat-template gemma
```

### 1.3 Installation Python (llama-cpp-python)

Dans le prototype présenté au chapitre 3, seul le module de benchmark (benchmark.py) s'appuie sur les bindings Python de llama.cpp ; le chatbot interactif (chatbot.py) pilote quant à lui directement le binaire en ligne de commande llama-cli en sous-processus, afin de contourner un problème de compatibilité de ces bindings sous Termux (chapitre 3, section 3.2.1). Ces bindings exposent l'ensemble des fonctionnalités du moteur C++ sous-jacent via une API Python simple, tout en conservant les performances natives de la bibliothèque.

L'installation standard fonctionne sur toute plateforme mais reste limitée au CPU. Sur Mac Apple Silicon, la variable d'environnement CMAKE\_ARGS permet de recompiler les bindings avec le support Metal au moment de l'installation ; de façon similaire, un GPU Nvidia sous Linux peut être exploité en activant le support CUDA. Le choix de la variante d'installation dépend donc uniquement du matériel de développement disponible, sans impact sur l'API Python exposée ensuite :

```
# Installation CPU (compatible partout)
pip install llama-cpp-python

# Installation avec support GPU Metal (Mac M1/M2/M3)
CMAKE_ARGS="-DGGML_METAL=ON" pip install llama-cpp-python

# Installation avec support CUDA (Linux + GPU Nvidia)
CMAKE_ARGS="-DGGML_CUDA=ON" pip install llama-cpp-python
```

### 1.4 Récapitulatif des performances mesurées (littérature)

Avant de présenter les mesures effectuées spécifiquement pour ce PIR (partie 3 de ce chapitre), il est utile de situer ces résultats par rapport aux performances rapportées dans la littérature et par les benchmarks communautaires sur des appareils comparables. Le tableau suivant rassemble des mesures de référence pour Gemma 2 2B en quantification Q4, sur smartphones, PC et Mac.

| Appareil      | SoC                | Modèle        | Prefill   | Decode    | RAM utilisée |
|---------------|--------------------|---------------|-----------|-----------|--------------|
| Xiaomi 14 Pro | Snapdragon 8 Gen 3 | Gemma 2 2B Q4 | ~28 tok/s | ~14 tok/s | ~2,4 Go      |
| Galaxy A54    | Exynos 1380        | Gemma 2 2B Q4 | ~10 tok/s | ~6 tok/s  | ~2,4 Go      |
| iPhone 15 Pro | Apple A17 Pro      | Gemma 2 2B Q4 | ~45 tok/s | ~22 tok/s | ~2,2 Go      |
| PC Ubuntu     | Intel i7-12th      | Gemma 2 2B Q4 | ~35 tok/s | ~18 tok/s | ~2,6 Go      |
| Mac M2        | Apple M2           | Gemma 2 2B Q4 | ~55 tok/s | ~28 tok/s | ~2,2 Go      |

**Tableau 2.4 :** Performances mesurées dans la littérature (Gemma 2 2B Q4, multi-plateformes)

*Sources : Xu et al. [22], Fassold [23], mesures directes sur appareils COTS.*

## Partie 2 : Déploiement via ML Kit GenAI / LiteRT

Cette seconde partie explore une voie alternative à llama.cpp : l'utilisation des API managées de Google, ML Kit GenAI et LiteRT, qui reposent sur le moteur AICore intégré nativement à certains appareils Android récents. Contrairement à llama.cpp, cette approche ne nécessite ni compilation ni gestion manuelle du modèle, mais impose en retour des contraintes de compatibilité matérielle strictes.

### 2.1 Vérification de compatibilité

ML Kit GenAI ne fonctionne que sur les appareils certifiés AICore par Google, ce qui exclut la grande majorité du parc Android existant. Il est donc indispensable de vérifier la compatibilité de l'appareil cible avant d'entamer le développement, à la fois au niveau matériel et au niveau des versions logicielles requises.

**2.1.1 Appareils certifiés AICore** Le SDK ML Kit GenAI expose une méthode dédiée permettant de vérifier, au runtime, si le modèle Gemini Nano est disponible sur l'appareil courant. Cette vérification doit être effectuée avant toute tentative d'inférence, faute de quoi l'application s'exposerait à une erreur d'exécution sur les appareils non supportés.

La fonction suivante interroge le client d'inférence pour connaître la disponibilité effective de la fonctionnalité sur l'appareil, indépendamment de la version d'Android installée. Cette vérification est asynchrone car elle peut nécessiter une communication avec les services Google Play. En cas d'indisponibilité ou d'erreur, la fonction retourne simplement false, ce qui permet à l'application de proposer un repli (par exemple vers llama.cpp) plutôt que de planter :

```
// Vérifier si l'appareil supporte ML Kit GenAI
import com.google.mlkit.genai.inference.LanguageModelInference
suspend fun checkDeviceSupport(): Boolean {
    return try {
        val availability = LanguageModelInference.getClient()
            .checkFeatureAvailability()
        availability == FeatureStatus.AVAILABLE
    } catch (e: Exception) {
        false
    }
}
```

**2.1.2 Versions Android/API requises** Au-delà de la certification matérielle, ML Kit GenAI impose des versions minimales d'Android, du SDK et des services Google Play. Le

tableau suivant résume les versions minimales et recommandées constatées lors de ce PIR.

| Condition            | Valeur      |
|----------------------|-------------|
| Android minimum      | 10 (API 29) |
| Android recommandé   | 14 (API 34) |
| ML Kit GenAI SDK     | 1.0.0-beta+ |
| Google Play Services | 24.20+      |

**Tableau 2.5 :** Versions Android et API requises pour ML Kit GenAI

## 2.2 Configuration du projet Android Studio

Cette section détaille la configuration d'un projet Android Studio destiné à intégrer ML Kit GenAI, depuis la création du projet jusqu'à la déclaration des permissions nécessaires. Ces étapes conditionnent le bon fonctionnement du SDK avant même d'écrire la moindre ligne de code d'inférence.

**2.2.1 Créer un nouveau projet** La création du projet suit le gabarit standard d'Android Studio, avec Kotlin comme langage principal. Le SDK minimum est fixé à l'API 29 (Android 10), condition nécessaire mais non suffisante pour bénéficier de ML Kit GenAI, qui reste par ailleurs restreint aux appareils certifiés AICore.

5. Ouvrir Android Studio (version Hedgehog 2023.1.1 ou plus récente)

6. File → New → New Project → Empty Views Activity

7. Paramètres :

- Name : LlmChatApp

- Package : com.PFE.llmchat

- Language : **Kotlin**

- Minimum SDK : **API 29 (Android 10)**

**2.2.2 Configurer build.gradle (Module: app)** Le fichier build.gradle.kts du module applicatif fixe le SDK minimum à l'API 29, condition requise par ML Kit GenAI, et cible le SDK 35 pour bénéficier des dernières API disponibles. Les dépendances ajoutées comprennent le module d'inférence ML Kit GenAI ainsi que les coroutines Kotlin, indispensables pour exécuter l'inférence de façon asynchrone sans bloquer le thread principal de l'interface. Le fichier complet, incluant la configuration de compilation et les

dépendances d'interface (RecyclerView, Material Design), est reproduit intégralement en Annexe A ; seul l'extrait le plus significatif est conservé ci-dessous :

```
dependencies {  
    implementation("com.google.mlkit:genai-inference:1.0.0-beta1")  
    implementation("org.jetbrains.kotlinx:kotlinx-coroutines-android:1.7.3")  
}
```

Un extrait du fichier build.gradle.kts est fourni en Annexe A ; le fichier complet est disponible sur le dépôt GitHub du projet.

**2.2.3 Configurer AndroidManifest.xml** Le manifeste Android déclare les permissions et métadonnées requises par ML Kit GenAI pour fonctionner correctement. La permission Internet est nécessaire car le modèle Gemini Nano, bien qu'exécuté localement, doit être téléchargé et mis à jour via les services Google Play lors de sa première utilisation. Le flag com.google.mlkit.genai.ENABLED doit également être déclaré explicitement pour activer la fonctionnalité au niveau de l'application :

```
<uses-permission android:name="android.permission.INTERNET" />  
  
<meta-data android:name="com.google.mlkit.genai.ENABLED" android:value="true" />
```

Un extrait du manifeste est fourni en Annexe B ; le fichier complet est disponible sur le dépôt GitHub du projet.

## 2.3 Code Kotlin : Inférence avec ML Kit GenAI

Cette section présente l'architecture logicielle de l'application de test : un ViewModel encapsulant la logique d'inférence, une activité principale gérant le cycle de vie et l'interface, et un layout XML définissant la disposition des éléments visuels. Cette séparation suit le patron MVVM (Model-View-ViewModel), standard sur Android, qui isole la logique métier de l'affichage.

**2.3.1 ViewModel (logique d'inférence)** La classe LlmViewModel encapsule l'ensemble de la logique d'inférence : elle maintient l'historique de conversation, gère les transitions d'état (chargement, génération, erreur) et pilote l'appel asynchrone à l'API ML Kit GenAI. Le prompt envoyé au modèle inclut les six derniers messages de l'historique afin de conserver un minimum de cohérence conversationnelle sans dépasser la fenêtre de contexte disponible. L'extrait ci-dessous illustre l'appel principal : la génération de la réponse se fait en streaming, avec accumulation progressive des tokens partiels et mesure de la latence totale une fois la réponse complète reçue :

```
client.generateResponseAsync(  
    prompt = buildPrompt(userInput),  
    options = options,  
    onPartialResult = { partial -> sb.append(partial) },
```

```
onComplete = { _ ->
val latency = System.currentTimeMillis() - startTime
// mise à jour de l'historique et de l'état d'inférence
}
)
```

Un extrait du ViewModel est fourni en Annexe C ; le code source complet (gestion de l'état, historique de conversation, construction du prompt) est disponible sur le dépôt GitHub du projet.

**2.3.2 MainActivity** MainActivity constitue le point d'entrée de l'application et orchestre l'ensemble de l'interface utilisateur. Elle initialise le modèle dès la création de l'activité, relie les actions de l'utilisateur (bouton d'envoi, validation clavier) à la logique du ViewModel, et observe en continu les flux d'état pour refléter à l'écran l'avancement de l'inférence. Cette observation repose sur les coroutines Kotlin et les StateFlow exposés par le ViewModel, un mécanisme réactif standard dans l'écosystème Android moderne. L'extrait ci-dessous montre uniquement la phase d'initialisation :

```
override fun onCreate(savedInstanceState: Bundle?) {
super.onCreate(savedInstanceState)
binding = ActivityMainBinding.inflate(layoutInflater)
setContentView(binding.root)
setupRecyclerView(); setupInput(); observeState()
viewModel.initializeModel()
}
```

Un extrait de MainActivity est fourni en Annexe D ; le code source complet (gestion du RecyclerView, de la saisie et des transitions d'état) est disponible sur le dépôt GitHub du projet.

**2.3.3 Layout XML (activity\_main.xml)** Le fichier de layout définit la structure visuelle de l'écran de conversation à l'aide d'un ConstraintLayout, qui permet de positionner les éléments les uns par rapport aux autres sans imbrication excessive de vues. Cette structure reste volontairement minimale afin de concentrer l'analyse de ce PIR sur la logique d'inférence plutôt que sur le raffinement de l'interface graphique.

L'interface repose sur un ConstraintLayout simple : une zone de statut en haut, une liste défilante des messages (RecyclerView) au centre, et une barre de saisie avec bouton d'envoi en bas. Un extrait du layout XML est fourni en Annexe E ; le fichier complet est disponible sur le dépôt GitHub du projet.

## 2.4 Cas d'usage avancés : Summarization et Proofreading

Au-delà de la génération de texte libre, ML Kit GenAI propose deux fonctionnalités spécialisées directement exploitables sans prompt engineering : le résumé automatique de texte et la correction grammaticale. Ces deux API illustrent l'intérêt des modèles on-device pour des tâches ciblées, où une interface dédiée et légère est préférable à un modèle conversationnel généraliste.

Les clients Summarization et Proofreading exposent chacun une méthode asynchrone unique, prenant un texte en entrée et retournant directement le résultat traité sans nécessiter de prompt spécifique ni de post-traitement. Cette simplicité d'usage contraste avec l'approche générative libre du ViewModel présenté en section 2.3, où la construction du prompt et le streaming de la réponse restent à la charge du développeur. Ces API sont particulièrement adaptées à des fonctionnalités ponctuelles intégrées dans une application existante (résumé de notes, relecture de messages), plutôt qu'à un assistant conversationnel complet :

```
import com.google.mlkit.genai.summarization.Summarization
import com.google.mlkit.genai proofreading.Proofreading
// Résumé d'un texte
suspend fun summarizeText(text: String): String {
    val client = Summarization.getClient()
    return client.summarize(text).await()
}
// Correction grammaticale
suspend fun proofreadText(text: String): String {
    val client = Proofreading.getClient()
    return client.proofread(text).await()
}
```

## 2.5 Limites de la solution ML Kit GenAI

Si ML Kit GenAI simplifie considérablement l'intégration d'un LLM dans une application Android, cette simplicité a une contrepartie : plusieurs contraintes structurelles limitent son usage à des scénarios précis. Le tableau suivant synthétise les principales limitations identifiées au cours de ce PIR.

| Contrainte                        | Impact                                          |
|-----------------------------------|-------------------------------------------------|
| Appareils certifiés seulement     | Exclut ~95 % du parc Android mondial            |
| Quota d'inférence par application | Impossibilité d'une utilisation intensive       |
| Exécution premier plan uniquement | Pas de traitement en arrière-plan               |
| Modèle non modifiable             | Impossible de fine-tuner ou d'adapter           |
| Dépendance Google totale          | Risque de dépréciation/modification unilatérale |

**Tableau 2.6 :** Limites de la solution ML Kit GenAI

**Conclusion :** *ML Kit GenAI est idéal pour les applications grand public ciblant les flagships récents. Pour un usage de recherche ou un déploiement sur un parc hétérogène, llama.cpp reste incontournable.*

## Partie 3 : Analyse des Performances

Cette troisième partie présente l'ensemble des mesures de performance réalisées dans le cadre de ce PIR, sur un corpus d'appareils Android couvrant plusieurs gammes et générations de SoC. Elle couvre le débit d'inférence (prefill et decode), la consommation énergétique, l'impact de la quantification et une comparaison entre plusieurs frameworks et environnements d'exécution.

### 3.1 Méthodologie de mesure

Toutes les mesures suivent le protocole défini par Xu et al. [22] et le profileur lm-Meter [24], qui distinguent quatre indicateurs complémentaires permettant de caractériser finement le comportement d'un LLM embarqué. Cette méthodologie est appliquée de façon identique à l'ensemble des appareils testés, afin de garantir la comparabilité des résultats présentés dans les sections suivantes :

- **Prefill** : traitement du prompt entrant (compute-bound, limité par la puissance CPU/NPU)
- **Decode** : génération token par token (memory-bound, limité par la bande passante RAM)
- **Throttling** : dégradation des performances mesurée après 5 minutes d'inférence continue
- **RAM delta** : mémoire supplémentaire consommée après chargement du modèle

Les mesures de ce chapitre ont été produites par deux scripts shell du dépôt du projet, qui s'appuient sur l'outil llama-bench de llama.cpp. Le script `benchmark_complet.sh` exécute llama-bench avec 512 tokens en prefill, 128 tokens en decode et cinq répétitions (-r 5), puis en extrait la moyenne et l'écart-type de chaque phase, la variation de RAM (RAM delta) et la variation de batterie avant/après. Le script `throttling_rigoureux.sh` mesure le throttling selon un protocole en cinq étapes : run de warm-up (pour neutraliser la montée en fréquence du governor schedutil), baseline de decode, charge continue de 5 minutes en boucle llama-bench à contexte fixe, mesure post-charge, puis dégradation calculée par  $(\text{baseline} - \text{post-charge}) / \text{baseline} \times 100$ . Aucune température n'est relevée : le throttling thermique est déduit de la variation du débit. Le module `benchmark.py` du prototype (chapitre 3, section 3.5) est un outil distinct, propre au prototype applicatif, qui n'a pas servi à produire les tableaux de ce chapitre :

# Benchmark complet (latence, RAM, batterie)

```
bash benchmark_complet.sh ~/models/Llama-3.2-1B-Instruct-Q4_K_M.gguf nom_appareil
# Test de throttling rigoureux (warm-up + 5 min de charge)
bash throttling_rigoureux.sh ~/models/Llama-3.2-1B-Instruct-Q4_K_M.gguf nom_appareil
```

### 3.2 Résultats de référence sur appareils réels (llama.cpp, Gemma 2 2B Q4\_K\_M)

Cette section présente les premières mesures obtenues avec llama.cpp sur cinq appareils représentatifs de gammes différentes, du flagship récent à l’entrée de gamme. Le modèle utilisé, Gemma 2 2B en quantification Q4\_K\_M, sert de référence commune pour comparer les SoC entre eux.

| SoC                | Appareil      | Prefill     | Decode      | RAM delta | Throttling 5 min |
|--------------------|---------------|-------------|-------------|-----------|------------------|
| Snapdragon 8 Gen 3 | Xiaomi 14 Pro | 28-35 tok/s | 12-16 tok/s | +2,4 Go   | −12 %            |
| Dimensity 9300     | Vivo X100     | 22-28 tok/s | 10-14 tok/s | +2,4 Go   | −14 %            |
| Apple A17 Pro      | iPhone 15 Pro | 40-52 tok/s | 18-24 tok/s | +2,2 Go   | −8 %             |
| Kirin 9000E        | Huawei P60    | 14-18 tok/s | 8-11 tok/s  | +2,5 Go   | −22 %            |
| Exynos 1380        | Galaxy A54    | 10-14 tok/s | 5-8 tok/s   | +2,4 Go   | −25 %            |

**Tableau 2.7 :** Résultats de référence sur appareils réels (llama.cpp, Gemma 2 2B Q4\_K\_M)

*Source : Xu et al. [22], Fassold [23], mesures protocole lm-Meter [24].*

**3.2.1 Mise en perspective avec la littérature récente** Les résultats obtenus dans ce PIR peuvent être mis en regard de deux études récentes portant sur des appareils et des modèles comparables. Cette comparaison permet de vérifier la cohérence des observations et d’identifier les phénomènes partagés entre plateformes, notamment en matière de throttling thermique.

**LLM Inference at the Edge** [30] (Tummalapalli et al., 2026) mesure un Galaxy S24 Ultra (Snapdragon 8 Gen 3) et un iPhone 16 Pro sous charge soutenue de 20 itérations avec Qwen 2.5 1.5B Q4. Résultat central : le GPU du S24 Ultra subit un arrêt complet de l’inférence lors des sessions prolongées, ce qui force un repli sur CPU. Ce phénomène est directement comparable au throttling thermique du Galaxy S26 (Snapdragon 8 Elite) mesuré dans ce PIR (−17,3 % en section 3.3). Les deux flagships Snapdragon partagent donc une vulnérabilité thermique sous charge soutenue qui n’est pas observée sur les appareils milieu de gamme testés (Snapdragon 730/778G, Dimensity 6400, Exynos 1280).

**PalmBench** [27] (Li et al., 2024) adopte une méthodologie similaire au protocole de ce PIR (llama.cpp, mesures prefill/decode répétées, charge soutenue), mais sur appareils Apple et Google Pixel uniquement. L’Exynos 1380 (Galaxy A54) apparaît dans le tableau de référence de Xu et al. [22] avec −25 % de throttling sur 5 min avec Gemma 2 2B ; nos mesures sur Exynos 1330 (Galaxy A16) et Exynos 1280 (Galaxy A26) avec Llama 3.2 1B

Q4\_K\_M (modèle plus léger) montrent respectivement  $-19,1$  % et aucun throttling, ce qui est cohérent avec l'impact de la taille de modèle sur la charge thermique.

### 3.3 Résultats mesurés en conditions réelles (protocole interne, llama.cpp, Llama 3.2 1B Q4\_K\_M)

Cette section présente le cœur des mesures originales de ce PIR : dix configurations combinant cinq appareils, deux environnements d'exécution (UserLand et Termux natif) et un modèle unique, Llama 3.2 1B Q4\_K\_M, choisi pour sa faible empreinte mémoire compatible avec l'ensemble du corpus testé. Chaque mesure est répétée plusieurs fois afin de fournir un écart-type représentatif de la stabilité du débit.

| SoC                      | Appareil           | Environnement | RAM totale | Prefill                  | Decode                  | Throttling                                              | Batterie        |
|--------------------------|--------------------|---------------|------------|--------------------------|-------------------------|---------------------------------------------------------|-----------------|
| Dimensity 6400 (6nm)     | Infinix Hot 60i 5G | UserLand      | 7625 Mo    | 54,78 $\pm$ 3,15 tok/s   | 11,86 $\pm$ 0,35 tok/s  | $-21,8$ % (artefact, pas de throttling)                 | $-1$ % / 3 min  |
| Dimensity 6400 (6nm)     | Infinix Hot 60i 5G | Termux natif  | 7625 Mo    | 54,75 $\pm$ 7,20 tok/s   | 12,67 $\pm$ 0,75 tok/s  | $-0,7$ % (artefact schedutil)                           | $-1$ % / 5 min  |
| Exynos 1280 (5nm)        | Galaxy A26         | UserLand      | 5427 Mo    | 65,21 $\pm$ 6,50 tok/s   | 6,86 $\pm$ 0,07 tok/s   | aucun (protocole 5 runs)                                | N/A             |
| Exynos 1280 (5nm)        | Galaxy A26         | Termux natif  | 5427 Mo    | 92,13 $\pm$ 30,42 tok/s  | 10,83 $\pm$ 2,54 tok/s  | $-100,2$ % (artefact governor)                          | $-1$ % / 15 min |
| Snapdragon 730 (8nm)     | Galaxy A71         | UserLand      | 7519 Mo    | 36,05 $\pm$ 0,07 tok/s   | 11,66 $\pm$ 0,03 tok/s  | $+0,3$ % (pas de throttling)                            | $-1$ % / 5 min  |
| Snapdragon 730 (8nm)     | Galaxy A71         | Termux natif  | 7519 Mo    | 46,66 $\pm$ 0,24 tok/s   | 11,40 $\pm$ 0,29 tok/s  | $-4,4$ % (artefact schedutil)                           | $-1$ % / 5 min  |
| Snapdragon 778G (6nm)    | Galaxy A73         | UserLand      | 7333 Mo    | 58,07 $\pm$ 14,09 tok/s  | 13,19 $\pm$ 0,13 tok/s  | $0,0$ % (aucun throttling)                              | $-2$ % / 5 min  |
| Snapdragon 778G (6nm)    | Galaxy A73         | Termux natif  | 7333 Mo    | 79,69 $\pm$ 1,07 tok/s   | 13,36 $\pm$ 0,80 tok/s  | $-104,8$ % (artefact governor)                          | $<1$ % / 5 min  |
| Exynos 1330 (5nm)        | Galaxy A16         | Termux natif  | 5452 Mo    | 57,99 $\pm$ 7,77 tok/s   | 14,00 $\pm$ 0,42 tok/s  | <b><math>-19,1</math> % (throttling thermique réel)</b> | $-3$ % / 12 min |
| Snapdragon 8 Elite (3nm) | Galaxy S26         | Termux natif  | 10240 Mo   | 235,65 $\pm$ 11,26 tok/s | 46,68 $\pm$ 14,40 tok/s | <b><math>-17,3</math> % (throttling thermique réel)</b> | $-2$ % / 12 min |

**Tableau 2.8 :** Résultats mesurés en conditions réelles (protocole interne, Llama 3.2 1B Q4\_K\_M)

**Points clés :**

- Le throttling thermique réel n'est confirmé que sur **Galaxy A16 (−19,1 %)** et **Galaxy S26 (−17,3 %)** ; les autres appareils restent dans l'enveloppe thermique sur 12 minutes.
- Les valeurs négatives extrêmes (−100 %, −104 %) sont des **artefacts du governor schedutil Android**, pas du throttling thermique (détaillés dans la section 3.8).
- Tous les appareils milieu de gamme atteignent **11-14 tok/s en decode**, dans la zone de fluidité conversationnelle.
- Ce débit dépasse nettement les 5-8 tok/s rapportés dans la littérature sur des SoC comparables avec Gemma 2 2B (tableaux 2.4 et 2.7) — un écart cohérent avec la nature memory-bound du decode (chapitre 1, section 1.4) : un modèle plus léger (Llama 3.2 1B, ~771 Mo) sollicite moins la bande passante mémoire à chaque token qu'un modèle plus lourd (Gemma 2 2B, ~1,5 Go), à SoC équivalent.

**3.4 Résultats Google on-device : Gemma 4 E2B-it via LiteRT (Galaxy S26)**

Cette section évalue la seconde voie de déploiement présentée en partie 2 de ce chapitre, cette fois au travers de l'application officielle Google AI Edge Gallery plutôt que d'un développement Android Studio complet. Le protocole reprend le même appareil et le même type de prompt que les mesures llama.cpp précédentes, afin de permettre une comparaison directe entre les deux approches.

*Le benchmark Google on-device a été réalisé via **AI Edge Gallery** (application officielle Google, Play Store), qui expose Gemma 4 E2B-it au format LiteRT quantifié (INT4, ~2,6 Go). Le modèle “via AICore” (Gemini Nano) s'est avéré indisponible sur le Galaxy S26 testé — non pas faute de certification matérielle (le Galaxy S26 figure sur la liste des appareils certifiés AICore par Google), mais parce que la fonctionnalité Gemini multi-app qui l'exploite est, à la date des tests, en déploiement bêta restreint à la Corée du Sud et aux États-Unis, anglais/coréen uniquement (Samsung, page d'assistance officielle TSG10010466) — une restriction géographique côté Google/Samsung, indépendante de l'appareil testé.*

**Protocole :** Galaxy S26 (Snapdragon 8 Elite, 3nm, 12 Go RAM) · Gemma 4 E2B-it LiteRT INT4 · Prompt : “Explique-moi le concept d’intelligence artificielle en 3 phrases.”

| Run            | Latence totale |
|----------------|----------------|
| 1              | 6,4 s          |
| 2              | 5,6 s          |
| 3              | 6,0 s          |
| <b>Moyenne</b> | <b>6,0 s</b>   |

**Tableau 2.9 :** Latence mesurée pour Gemma 4 E2B-it via AI Edge Gallery (Galaxy S26)

Débit estimé : ~70 tokens (3 phrases) → **≈ 11-12 tok/s.**

**Comparaison avec llama.cpp sur le même appareil :**

| Critère                | llama.cpp (Llama 3.2 1B Q4_K_M)                                                                        | AI Edge Gallery<br>(Gemma 4 E2B<br>LiteRT) |
|------------------------|--------------------------------------------------------------------------------------------------------|--------------------------------------------|
| Modèle                 | 1B paramètres, ~800 Mo                                                                                 | 2B paramètres, 2,6 Go                      |
| Decode (tok/s)         | 46,68 ± 14,40 tok/s (llama-bench) ; 55,4 tok/s (prototype, 3 runs)                                     | ~11-12 tok/s (estimé)                      |
| Latence réponse courte | 3,1 s en moyenne (2,7 à 3,3 s, 3 runs), dont ~1,0 s de rechargement du modèle ; ~2,0 s hors chargement | ~6,0 s                                     |
| Throttling             | ~17,3 % (thermique réel)                                                                               | non mesuré                                 |
| Installation           | Termux + wget (~10 min)                                                                                | Play Store + téléch. 2,6 Go                |

**Tableau 2.10 :** Comparaison llama.cpp vs AI Edge Gallery sur le même appareil (Galaxy S26)

**Interprétation :** la latence supérieure du modèle LiteRT s’explique par la taille du modèle (2B vs 1B), le format d’inférence (LiteRT CPU vs llama.cpp CPU avec optimisations BLAS), et l’absence d’accélération NPU sur ce chemin. Dans nos conditions, llama.cpp (Llama 3.2 1B) est environ 2 fois plus rapide en latence totale (3,1 s contre 6,0 s), et environ 3 fois hors rechargement du modèle (2,0 s), que LiteRT (Gemma 4 E2B, ~2B) sur le même SoC ; cet écart compare deux modèles de tailles différentes et ne peut donc pas être attribué au seul framework. De plus, le débit LiteRT (~11-12 tok/s) est une estimation calculée sur la latence totale (~70 tokens en 6,0 s, prefill et démarrage inclus), alors que celui de llama.cpp est un débit de decode mesuré : les deux valeurs ne sont pas strictement comparables.

**Protocole de la mesure llama.cpp :** prototype chatbot.py (chapitre 3), Termux natif, Llama 3.2 1B Q4\_K\_M, build b10154-0e4a03622, session neuve à chaque run (contexte vide), même prompt que pour LiteRT, 3 runs : 3,1 s, 3,3 s et 2,7 s (moyenne 3,1 s) ; decode de 55,3 à 55,4 tok/s ; prefill de 216 à 218 tok/s. Cette latence inclut le rechargement du

modèle à chaque tour de parole ( $\sim 1,0$  s), propre à l'architecture du prototype (chapitre 3, section 3.2.1) ; hors rechargement, elle est d'environ 2,0 s.

### 3.5 Benchmark Gemini 2.0 Flash API (cloud) vs on-device

Après avoir comparé deux solutions on-device, cette section introduit un troisième point de référence : l'API cloud Gemini 2.0 Flash, appelée directement depuis le même appareil et sur le même réseau Wi-Fi. Cet ajout permet de quantifier concrètement l'écart de latence entre une inférence locale et un appel à un service distant, dans des conditions réseau réalistes plutôt que théoriques.

Mesure de la latence de l'API cloud Gemini 2.0 Flash depuis Termux sur le Galaxy S26 (connexion Wi-Fi), même prompt que les tests on-device.

| Run                             | Temps total    |
|---------------------------------|----------------|
| 1 (cold start, TLS + connexion) | 2,022 s        |
| 2                               | 0,286 s        |
| 3                               | 0,298 s        |
| <b>Moyenne warm (runs 2-3)</b>  | <b>0,292 s</b> |

**Tableau 2.11** : Latence de l'API Gemini 2.0 Flash depuis le Galaxy S26

| Solution                    | Type         | Modèle                  | Latence (réponse courte)                       |
|-----------------------------|--------------|-------------------------|------------------------------------------------|
| llama.cpp (Termux)          | On-device    | Llama 3.2 1B Q4_K_M     | $\sim 3,1$ s ( $\sim 2,0$ s hors rechargement) |
| AI Edge Gallery             | On-device    | Gemma 4 E2B LiteRT INT4 | $\sim 6,0$ s (moy.)                            |
| <b>Gemini 2.0 Flash API</b> | <b>Cloud</b> | <b>gemini-2.0-flash</b> | <b>0,29 s (warm)</b>                           |

**Tableau 2.12** : Synthèse on-device vs cloud (Galaxy S26, même prompt)

L'API cloud est  **$\sim 20\times$  plus rapide** que la solution LiteRT et  **$\sim 7\times$  (hors rechargement du modèle) à  $\sim 10\times$  (avec rechargement) plus rapide** que llama.cpp sur ce même appareil. Ce résultat résume le compromis de l'inférence embarquée : latence cloud minimale mais dépendance réseau et transmission des données à des serveurs externes ; on-device plus lent mais confidentialité totale et fonctionnement hors ligne.

### 3.6 Comparaison llama.cpp vs MLC-LLM (Snapdragon 8 Gen 3)

MLC-LLM se distingue de llama.cpp par une approche de compilation spécifique à chaque cible matérielle via Apache TVM, ce qui lui permet notamment d'exploiter le GPU Mali via Vulkan. Cette section compare directement les deux frameworks sur un même appareil

(Snapdragon 8 Gen 3) afin de quantifier le gain de performance apporté par cette compilation ciblée.

| Métrique          | llama.cpp | MLC-LLM      | Gain MLC |
|-------------------|-----------|--------------|----------|
| Prefill (tok/s)   | 28-35     | 34-44        | +23 %    |
| Decode (tok/s)    | 12-16     | 15-20        | +25 %    |
| Latence 1er token | 0,8-1,2s  | 0,6-0,9s     | -25 %    |
| RAM utilisée      | 2,4 Go    | 2,3 Go       | -4 %     |
| GPU Mali activé   | Non       | Oui (Vulkan) | —        |

**Tableau 2.13** : Comparaison llama.cpp vs MLC-LLM (Snapdragon 8 Gen 3)

**Conclusion** : MLC-LLM est 20-25 % plus rapide grâce à l’optimisation compilateur TVM et au support GPU Mali via Vulkan.

### 3.7 Impact de la quantification sur la qualité

La quantification réduit la taille du modèle et accélère l’inférence, mais dégrade en contrepartie la qualité des réponses générées. Cette section quantifie précisément ce compromis sur Gemma 2 2B, en comparant cinq niveaux de quantification depuis la référence FP16 jusqu’à l’extrême Q2\_K.

| Format           | Taille (Gemma 2 2B) | MMLU          | GSM8K         | Decode (Snap. 8 Gen 3) |
|------------------|---------------------|---------------|---------------|------------------------|
| FP16 (référence) | 4,8 Go              | 52,4 %        | 46,2 %        | 6-8 tok/s              |
| Q8_0             | 2,4 Go              | 52,1 %        | 45,8 %        | 11-14 tok/s            |
| <b>Q4_K_M</b>    | <b>1,6 Go</b>       | <b>51,6 %</b> | <b>45,1 %</b> | <b>12-16 tok/s</b>     |
| Q3_K_M           | 1,2 Go              | 49,8 %        | 42,3 %        | 14-18 tok/s            |
| Q2_K             | 0,9 Go              | 45,1 %        | 36,7 %        | 16-20 tok/s            |

**Tableau 2.14** : Impact du format de quantification sur la qualité et la vitesse (Gemma 2 2B)

**Sweet spot validé** : Q4\_K\_M offre -1 % de qualité vs FP16 pour -67 % de taille.

**Validation indépendante** : Song et al. [29] établissent un seuil critique à 3,5 BPW en dessous duquel la qualité chute significativement sur 7 méthodes PTQ et des modèles de 0,5B à 14B. Le format Q4\_K\_M (~4,5 BPW) est au-dessus de ce seuil, ce qui valide indépendamment le choix de quantification de ce PIR.

### 3.8 Consommation énergétique

Le débit d’inférence ne constitue qu’un axe d’évaluation parmi d’autres : la consommation énergétique conditionne directement l’autonomie de l’appareil lors d’un usage prolongé.

Cette section distingue les données rapportées par la littérature de celles mesurées spécifiquement dans le cadre de ce PIR.

**3.8.1 Données de référence (littérature)** Le tableau suivant rassemble des mesures de consommation batterie rapportées pour différents SoC et différents modes d'exécution (CPU via llama.cpp, NPU via AICore, Neural Engine Apple). Ces valeurs de référence permettent de situer les mesures internes présentées dans la section suivante.

| SoC                | Consommation batterie | Mode                     |
|--------------------|-----------------------|--------------------------|
| Snapdragon 8 Gen 3 | 5-7 % / 10 min        | CPU llama.cpp            |
| Snapdragon 8 Gen 3 | 3-4 % / 10 min        | NPU AICore (Gemini Nano) |
| Exynos 1380        | 8-12 % / 10 min       | CPU llama.cpp            |
| Apple A17 Pro      | 4-6 % / 10 min        | Neural Engine            |

**Tableau 2.15 :** Consommation énergétique de référence (littérature)

Source : Xu et al. [22], mesures protocole lm-Meter [24].

**3.8.2 Données mesurées (protocole interne, Llama 3.2 1B Q4\_K\_M, ~12 min)** Les mesures suivantes ont été réalisées sur le corpus d'appareils de ce PIR, sur des sessions d'environ douze minutes d'inférence continue avec Llama 3.2 1B Q4\_K\_M. Le delta de batterie est mesuré directement via les outils système d'Android, avant et après la session de test.

| SoC                      | Appareil                    | $\Delta$ batterie | Durée   |
|--------------------------|-----------------------------|-------------------|---------|
| Dimensity 6400 (6nm)     | Infinix Hot 60i 5G (Termux) | -2 %              | ~12 min |
| Snapdragon 730 (8nm)     | Galaxy A71 (UserLAnd)       | -3 %              | ~12 min |
| Snapdragon 730 (8nm)     | Galaxy A71 (Termux)         | -2 %              | ~12 min |
| Snapdragon 778G (6nm)    | Galaxy A73 (Termux)         | -4 %              | ~12 min |
| Exynos 1330 (5nm)        | Galaxy A16 (Termux)         | -3 %              | ~12 min |
| Snapdragon 8 Elite (3nm) | Galaxy S26 (Termux)         | -2 %              | ~12 min |

**Tableau 2.16 :** Consommation énergétique mesurée (protocole interne, Llama 3.2 1B Q4\_K\_M)

Tous les appareils se situent entre **-2 % et -4 % / 12 min**, nettement inférieur aux données littérature sur Gemma 2 2B (5-12 %), ce qui confirme l'impact majeur de la taille du modèle sur la consommation.

### 3.9 Latence perçue et seuils d'acceptabilité

Au-delà des chiffres bruts de débit, il est utile de relier la vitesse de décodage mesurée à la perception subjective de fluidité par un utilisateur final. Le tableau suivant propose

une grille de lecture inspirée des seuils communément admis pour les interfaces conversationnelles.

| Vitesse de décodage | Ressenti utilisateur                       |
|---------------------|--------------------------------------------|
| < 5 tok/s           | Inacceptable                               |
| 5-10 tok/s          | Acceptable pour la lecture                 |
| <b>10-20 tok/s</b>  | <b>Fluide pour le chat conversationnel</b> |
| > 20 tok/s          | Excellent                                  |

**Tableau 2.17** : Seuils d’acceptabilité perçue selon la vitesse de décodage

Tous les appareils milieu de gamme testés (11-14 tok/s) se situent dans la zone fluide.

### 3.10 Limitations matérielles identifiées

Les mesures précédentes mettent en évidence trois limitations matérielles récurrentes, indépendantes du framework utilisé. Cette section les détaille individuellement, avec les mitigations envisageables lorsqu’elles existent.

**3.10.1 Throttling thermique** Le throttling thermique correspond à la réduction automatique de la fréquence du processeur par le système d’exploitation lorsque la température de l’appareil dépasse un seuil de sécurité. Sur le corpus testé, ce phénomène reste limité à deux appareils spécifiques.

- **Exynos 1330, Galaxy A16** : -19,1 % après ~8 min. Seul cas réel dans le corpus milieu de gamme.
- **Snapdragon 8 Elite, Galaxy S26** : -17,3 % après ~12 min. Cohérent avec [30] (S24 Ultra).
- **Tous les autres appareils** : aucun throttling thermique sur 12 min avec le modèle 1B.

**Mitigation** : sur Exynos 1330 (Galaxy A16), limiter les sessions à 5-6 minutes.

**3.10.2 Contrainte de bande passante mémoire** Contrairement au prefill, qui sollicite principalement la puissance de calcul brute, le decode token par token est structurellement limité par la vitesse à laquelle les poids du modèle peuvent être lus depuis la mémoire. Cette contrainte explique l’essentiel de l’écart de performance observé entre générations de mémoire LPDDR.

Le décodage est fondamentalement limité par la bande passante RAM :

- LPDDR5X (77 GB/s) → 12-16 tok/s pour Gemma 2 2B
- LPDDR4X (34 GB/s) → 5-8 tok/s pour le même modèle

**3.10.3 Incompatibilité GPU Mali avec llama.cpp** Le GPU Mali, présent sur la quasi-totalité des SoC Samsung Exynos et MediaTek Dimensity, n'est pas exploité par llama.cpp faute de support Vulkan natif dans ce framework. Cette limitation cantonne l'inférence à une exécution CPU pure sur une large partie du parc Android non-Snapdragon.

Sur les appareils Samsung Exynos et MediaTek Dimensity, llama.cpp ne peut pas exploiter le GPU Mali. Seul MLC-LLM (via Vulkan) résout ce problème, au prix d'une recompilation du modèle par appareil.

### 3.11 Comparaison UserLand vs Termux natif

UserLand et Termux constituent deux façons distinctes d'exécuter un environnement Linux sur Android, avec des implications différentes sur la performance et la reproductibilité des mesures. Cette section compare directement les deux environnements sur les quatre appareils où les deux configurations ont pu être testées.

| Appareil           | Decode UserLand        | Decode Termux          | $\Delta$                  |
|--------------------|------------------------|------------------------|---------------------------|
| Infinix Hot 60i 5G | 11,86 $\pm$ 0,35 tok/s | 12,67 $\pm$ 0,75 tok/s | +6,8 % Termux             |
| Galaxy A26         | 6,86 $\pm$ 0,07 tok/s  | 10,83 $\pm$ 2,54 tok/s | +57,9 % Termux (anomalie) |
| Galaxy A71         | 11,66 $\pm$ 0,03 tok/s | 11,40 $\pm$ 0,29 tok/s | -2,3 % UserLand           |
| Galaxy A73         | 13,19 $\pm$ 0,13 tok/s | 13,36 $\pm$ 0,80 tok/s | +1,3 % Termux             |

**Tableau 2.18** : Comparaison des performances decode : UserLand vs Termux natif

**Variance** : UserLand présente une variance prefill systématiquement plus faible que Termux natif. La couche proot isole les processus du governor schedutil Android, ce qui donne des mesures plus reproductibles. Termux natif offre de meilleures performances brutes, mais avec une variance plus élevée.

**Recommandation** : Termux natif pour un usage applicatif (performances brutes) ; UserLand pour la recherche et la reproductibilité des benchmarks.

### 3.12 Discussion et conclusion de l'analyse

Les résultats confirment que l'inférence LLM on-device est **techniquement viable sur smartphone Android milieu de gamme** en 2025-2026, sous deux conditions : un modèle  $\leq 2B$  paramètres en quantification Q4, et un déploiement via Termux natif.

**Performances** : les appareils milieu de gamme testés (Snapdragon 730/778G, Exynos 1280/1330, Dimensity 6400) atteignent 11-14 tok/s en decode, dans la zone de fluidité conversationnelle ; le Snapdragon 8 Elite (Galaxy S26 Ultra), seul représentant haut de gamme du corpus, atteint un débit nettement supérieur (46,68  $\pm$  14,40 tok/s), largement au-delà de ce seuil.

**Throttling** : la littérature documente un throttling thermique réel sur les appareils haut de gamme (−8 à −25 % sur 5 min avec Gemma 2 2B). Notre corpus le confirme sur son propre représentant haut de gamme, le Snapdragon 8 Elite (Galaxy S26 Ultra, −17,3 % après ~12 min), mais montre aussi que ce phénomène n’est pas réservé au haut de gamme : l’Exynos 1330 (Galaxy A16), pourtant milieu de gamme, présente lui aussi un throttling thermique réel (−19,1 % après ~8 min) — le seul cas de ce type observé hors segment haut de gamme dans notre corpus.

**Consommation** : le modèle 1B Q4 consomme 2–4 % de batterie par 12 minutes de charge, un niveau acceptable pour un usage applicatif réel (sessions de 1–5 minutes).

Ces résultats montrent que le LLM embarqué est une alternative crédible aux API cloud pour des usages conversationnels légers sur un smartphone récent milieu de gamme, sans dépendance réseau et avec une confidentialité totale des données.

## Annexes

### Annexes A à E : Code source de l’application Android (ML Kit GenAI / Gemini Nano)

Le code source complet de l’application de démonstration décrite dans la partie 2 de ce chapitre (sections 2.2 et 2.3) est publié sur le dépôt public du projet, qui constitue la référence à jour et compilable :

git clone <https://github.com/on-device-llm/llm-smartphone>

Ce dépôt est privilégié ici à une reproduction intégrale du code, dont la lecture sur papier n’apporte pas de valeur supplémentaire et qui deviendrait obsolète à chaque évolution des bibliothèques ML Kit GenAI (encore en version bêta). Seuls quelques extraits significatifs sont conservés dans les annexes ci-dessous ; les fichiers concernés se trouvent dans le répertoire `prototype-android/` du dépôt :

- `prototype-android/app/build.gradle.kts` (section 2.2.2, Annexe A) : configuration du module, SDK minimum et dépendances ML Kit GenAI.
- `prototype-android/app/src/main/AndroidManifest.xml` (section 2.2.2, Annexe B) : permissions et déclaration de l’activité principale.
- `prototype-android/app/src/main/java/com/pfe/llmchat/LlmViewModel.kt` (section 2.3.1, Annexe C) : gestion de l’état d’inférence, historique de conversation et construction du prompt.
- `prototype-android/app/src/main/java/com/pfe/llmchat/MainActivity.kt` (section 2.3.2, Annexe D) : liaison de l’interface, saisie et observation de l’état.
- `prototype-android/app/src/main/res/layout/activity_main.xml` (section 2.3.3, Annexe E) : disposition de l’écran de conversation.

### Annexe A : Configuration Gradle du projet Android (`build.gradle.kts`, extrait)

```
android {
```

```
compileSdk = 35
defaultConfig {
    minSdk = 29
    targetSdk = 35
}
}
dependencies {
    implementation("com.google.mlkit:genai-common:1.0.0-beta1")
    implementation("com.google.mlkit:genai-inference:1.0.0-beta1")
    // [...] coroutines, lifecycle, RecyclerView, Material
}
```

**Annexe B : Manifeste Android** (AndroidManifest.xml, extrait)

```
<!-- Requis pour télécharger le modèle Gemini Nano -->
<uses-permission android:name="android.permission.INTERNET" />
<!-- [...] déclaration de l'activité principale -->
<meta-data
    android:name="com.google.mlkit.genai.ENABLED"
    android:value="true" />
```

**Annexe C : Logique d'inférence** (LlmViewModel.kt, extrait)

```
fun initializeModel() {
    viewModelScope.launch {
        _state.value = InferenceState.ModelLoading
        try {
            val availability = LanguageModelInference.checkAvailability()
            if (!availability.isAvailable) {
                _state.value = InferenceState.Error(
                    "Gemini Nano non disponible sur cet appareil. " +
                    "Appareils supportés : Pixel 9/10, Galaxy S25/S26.")
                return@launch
            }
            modelClient = LanguageModelInference.getClient()
            _state.value = InferenceState.ModelReady
        } // [...]
    }
}
```

**Annexe D : Activité principale** (MainActivity.kt, extrait)

```
override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    binding = ActivityMainBinding.inflate(layoutInflater)
    setContentView(binding.root)
    setupRecyclerView()
    setupInput()
    observeState()
}
```

```
viewModel.initializeModel()  
}
```

**Annexe E : Layout XML** (activity\_main.xml, extrait)

```
<androidx.constraintlayout.widget.ConstraintLayout ...>  
    <TextView  
        android:id="@+id/tv_status"  
        android:text="Initialisation..." ... />  
    <androidx.recyclerview.widget.RecyclerView  
        android:id="@+id/rv_chat" ... />  
    <!-- [...] zone de saisie (TextInputEditText + bouton Envoyer) -->  
</androidx.constraintlayout.widget.ConstraintLayout>
```

Le code complet de chacun de ces fichiers, y compris la classe ChatAdapter non reproduite ici, est disponible sur le dépôt indiqué ci-dessus.